Assignment 2

Frontend — HTML, CSS & JavaScript: Questions & Answers

Question 1

Write CSS Flexbox code for a navbar with a logo on the left and 3 menu links on the right, all vertically centered.

HTML:

```
<nav class="navbar">
  <div class="logo">MyLogo</div>
  <div class="nav-links">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </div>
</nav>
```

CSS:

```
.navbar {
  display: flex;
  justify-content: space-between; /* Logo left me, links right me */
  align-items: center;            /* Vertically center align */
  padding: 1rem 2rem;
  background-color: #333;
  color: white;
}

.nav-links {
  display: flex;
  gap: 20px;
}

.nav-links a {
  color: white;
  text-decoration: none;
}
```

Question 2

This button is supposed to toggle a "dark-mode" class on the body when clicked, but nothing happens. Find and fix the bug.

```
const button = document.querySelector('.toggle-btn');
button.addEventListner('click', () => {
  document.body.classList.toggle('dark-mode');
});
```

Bug:

`addEventListener` ki spelling galat likhi hai (`addEventListner` me 'e' missing hai).

Corrected Code:

```
const button = document.querySelector('.toggle-btn');
button.addEventListener('click', () => {
  document.body.classList.toggle('dark-mode');
});
```

Question 3

Write vanilla JavaScript to validate a signup form: check that the email contains "@" and the password is at least 8 characters, before allowing submission — without using any library.

Answer:

```
const form = document.querySelector('#signupForm');

form.addEventListener('submit', (e) => {
  const email = document.querySelector('#email').value;
  const password = document.querySelector('#password').value;

  if (!email.includes('@')) {
    e.preventDefault();
    alert('Please enter a valid email containing "@".');
    return;
  }

  if (password.length < 8) {
    e.preventDefault();
    alert('Password must be at least 8 characters long.');
    return;
  }
});
```

Question 4

This code attaches a new scroll event listener every time `renderPage()` runs, without ever removing the old one. In 2 lines, explain why this causes a memory leak, then rewrite it correctly.

```
function renderPage() {
  window.addEventListener('scroll', () => {
    console.log(window.scrollY);
  });
}
renderPage();
renderPage();
```

Why it causes a memory leak:

Multiple scroll listeners accumulate in memory every time `renderPage()` runs, executing redundant callbacks simultaneously. These references remain bound to the global `window` object, preventing garbage collection.

Corrected Code:

```
function handleScroll() {
  console.log(window.scrollY);
}

function renderPage() {
  window.removeEventListener('scroll', handleScroll);
  window.addEventListener('scroll', handleScroll);
}

renderPage();
renderPage();
```

Question 5

Write a debounced search-input handler in JavaScript that waits 400ms after the user stops typing before calling an API function `searchProducts(query)`.

Answer:

```
function debounce(func, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => func.apply(this, args), delay);
  };
}
```

```
// Handler setup
const searchInput = document.querySelector('#searchInput');

const handleSearch = debounce((e) => {
  searchProducts(e.target.value);
}, 400);

searchInput.addEventListener('input', handleSearch);
```

Question 6

This layout breaks below 480px screen width. Add a media query that fixes it for mobile, and explain in 1 line why the original CSS breaks.

```
.container {
  display: flex;
  width: 960px;
}
.card {
  width: 300px;
}
```

Why it breaks:

The container has a fixed hardcoded width of 960px, which exceeds small mobile viewports and causes horizontal overflow.

Media Query Fix:

```
@media (max-width: 480px) {
  .container {
    width: 100%;
    flex-direction: column;
  }
  .card {
    width: 100%;
  }
}
```

Question 7

Write JavaScript code using localStorage to save and restore a user's selected theme ("light" or "dark") so it persists across page reloads.

Answer:

```
// Function to apply saved theme on initial page load
function loadSavedTheme() {
  const savedTheme = localStorage.getItem('userTheme') || 'light';
  document.body.classList.add(savedTheme);
}

// Function to toggle and store theme
function toggleTheme() {
  if (document.body.classList.contains('dark')) {
    document.body.classList.replace('dark', 'light');
    localStorage.setItem('userTheme', 'light');
  } else {
    document.body.classList.replace('light', 'dark');
    localStorage.setItem('userTheme', 'dark');
  }
}

// Initial Call
loadSavedTheme();
```

Question 8

This form refreshes the page on submit and loses all JS state. Rewrite the code to prevent the default behavior and submit the data using `fetch()` instead.

```
<form id="contactForm">
  <input name="email" />
  <button type="submit">Send</button>
</form>
```

Answer:

```
const form = document.querySelector('#contactForm');

form.addEventListener('submit', async (e) => {
  e.preventDefault(); // Prevents page reload

  const formData = new FormData(form);
  const data = Object.fromEntries(formData.entries());

  try {
    const response = await fetch('/api/contact', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(data),
    });

    const result = await response.json();
    console.log('Submission successful:', result);
  } catch (error) {
    console.error('Error submitting form:', error);
  }
});
```

Question 9

This async function is meant to return the parsed user data, but callers receive a pending Promise instead. Identify the missing keyword and correct the code.

```
async function getUser(id) {
  const res = fetch(`/api/users/${id}`);
  const data = res.json();
  return data;
}
```

Missing Keyword:

The `await` keyword was missing before the asynchronous operations (`fetch()` and `res.json()`).

Corrected Code:

```
async function getUser(id) {
  const res = await fetch(`/api/users/${id}`);
  const data = await res.json();
  return data;
}
```

Question 10

Write CSS Grid code for a responsive image gallery with 4 columns on desktop and 2 columns on mobile, using `minmax()` and `auto-fit` instead of a media query.

Answer:

```
.gallery {
  display: grid;
  /* Auto-fits items with a minimum width and scales smoothly */
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
```

```
}

.gallery img {
  width: 100%;
  height: auto;
  display: block;
}
```